

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

IDE 反汇编实验

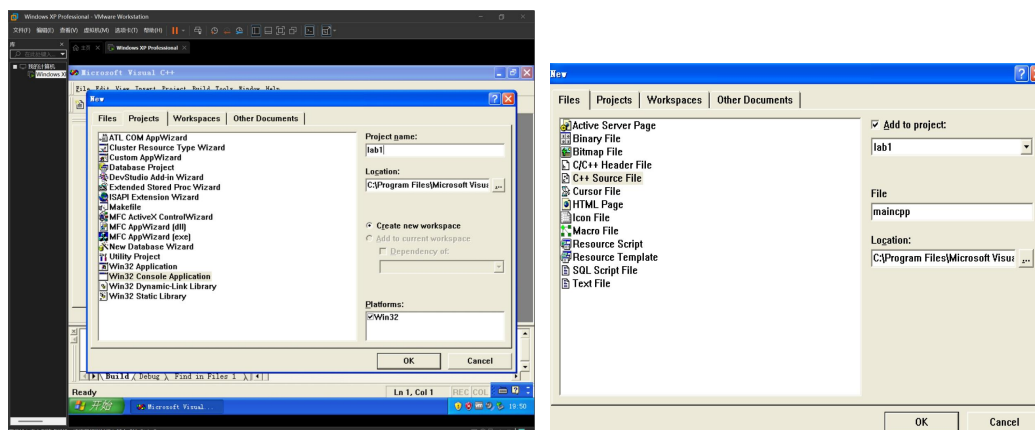
实验要求：

根据第二章示例 2-1，在 XP 环境下进行 VC6 反汇编调试，熟悉函数调用、栈帧切换、CALL 和 RET 指令等汇编语言实现，将 call 语句执行过程中的 EIP 变化、ESP、EBP 变化等状态进行记录，解释变化的主要原因。

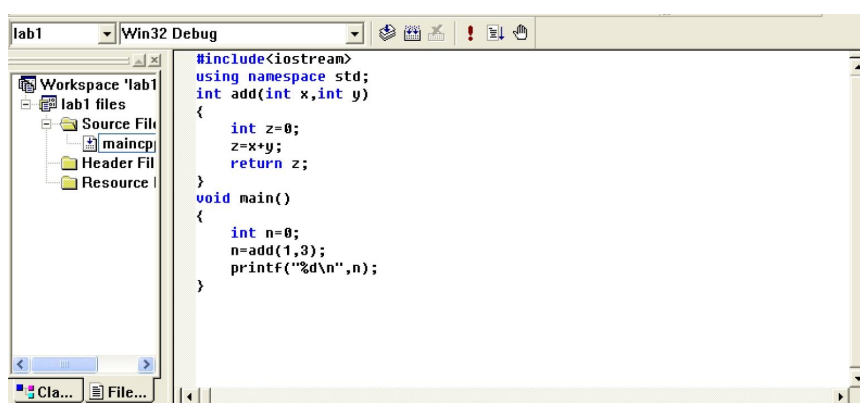
实验过程：

1. 进入 VC 反汇编

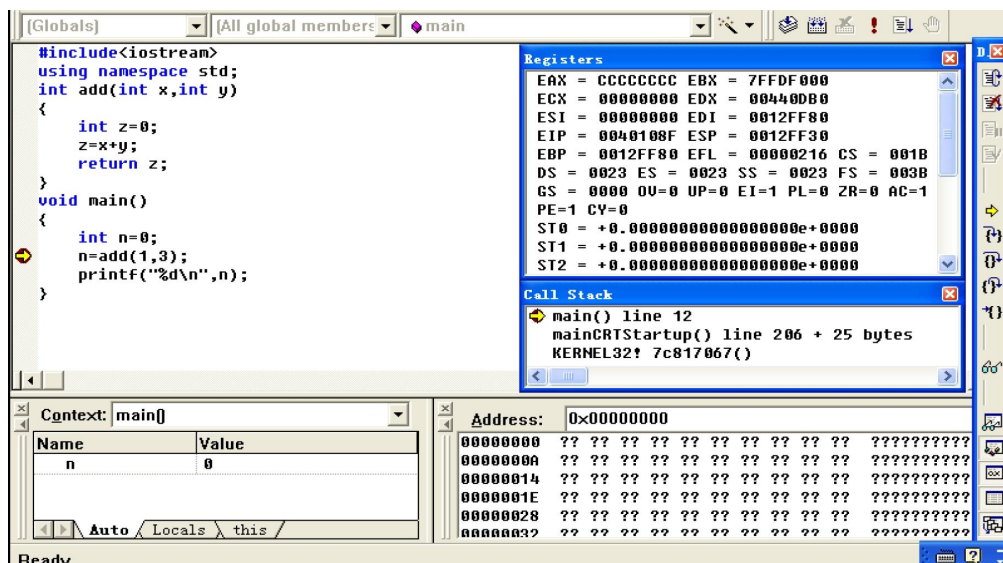
打开 vmware 软件，启动 XP 系统虚拟机，打开 Microsoft Visual C++, 新建一个项目和一个空 C++ 文件。



输入主要代码，经过编译和运行检查无误后，进行后续操作。



在代码 `n=add(1,3)` 处设置断点，按 F5 启用调试，发现程序执行到该代码处停下，接着选择“Go To Disassembly”反汇编，观察和分析汇编代码。



2. 观察 add 函数调用前后语句

2.1>调用前

```

11:      int n=0;
00401088 mov     dword ptr [ebp-4],0
12:      n=add(1,3);

```

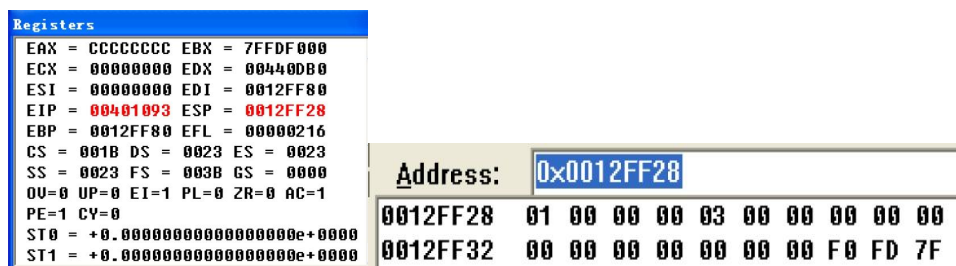
这是一个 MOV 指令，为局部变量 n 分配了一定的内存空间，ebp-4 表示将 ebp 寄存器抬高 4 字节，存储变量 n 并初始化值为 0。

```

12:      n=add(1,3);
0040108F push    3
00401091 push    1
00401093 call    @ILT+0(add) (00401005)

```

按下 F10，指令从断点处继续执行。两条 PUSH 指令将两个参数 1 和 3 从右到左依次入栈。



可以看到 esp 从 0012FF30 变为 0012FF28，表明两个参数入栈成功（入栈由高地址向低地址方向增长），从 Address 输入查询，也可以看到 1 和 3 成功存入。

```

00401093 call    @ILT+0(add) (00401005)

```

接下来调用了 CALL 指令，将会跳转调用 ADD 函数。00401005 表明将会跳转到地址为 00401005 的语句，按 F11 将会步入 ADD 函数内部，继续跟踪。

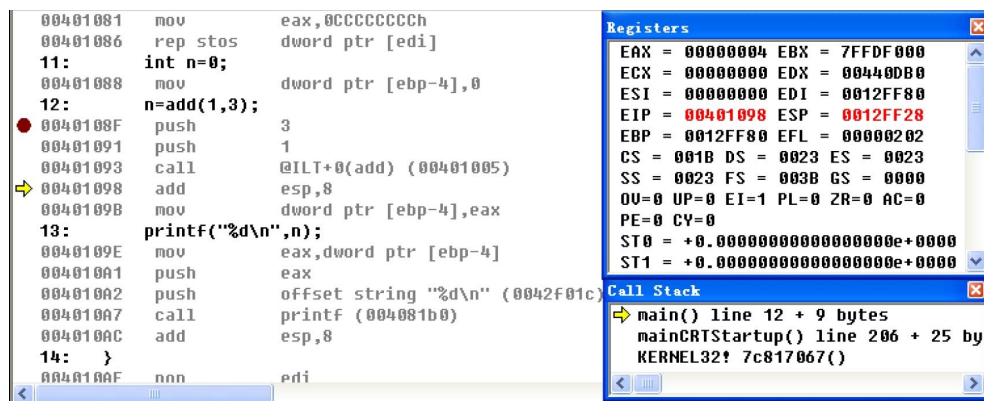
```

@ILT+0(?add@?YAHHHQZ) :
00401005 jmp     add (00401030)

```

发现语句继续跳转，跳到 00401030 地址处的语句，即 ADD 函数入口。

2.2>调用后



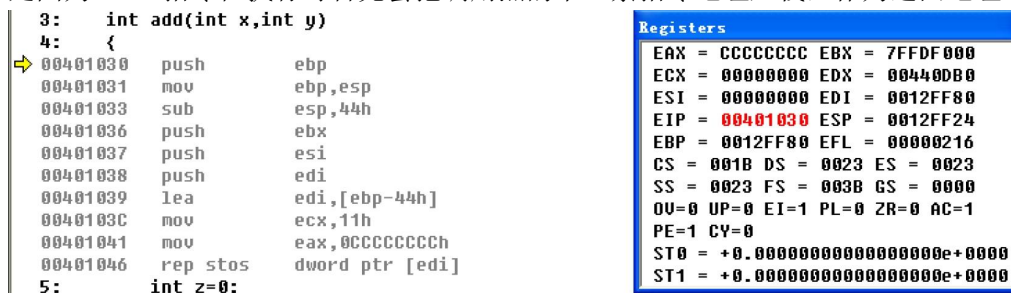
esp 回到了 0012FF28;接着 add 语句将 esp 寄存器位置下移 8 个字节, 旨在恢复调用前所占用的空间。



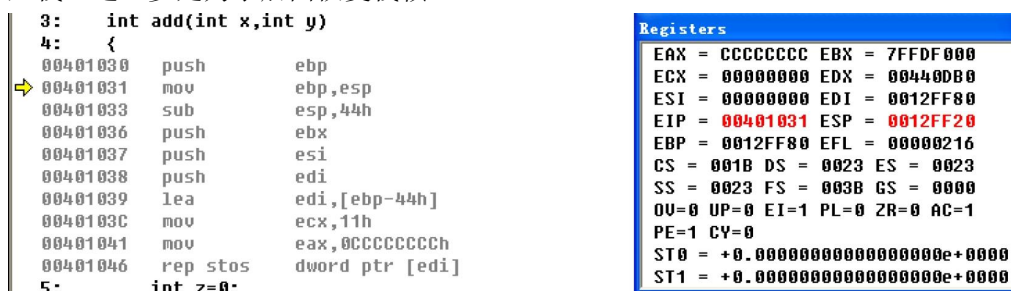
eax 寄存器中存储着 ADD 函数返回的值(即 1+3 的结果 4), 用 mov 指令将该计算结果赋值给变量 n (ebp-4 即 0012FF7C 处)。

3. add 函数内部栈帧切换等关键汇编代码

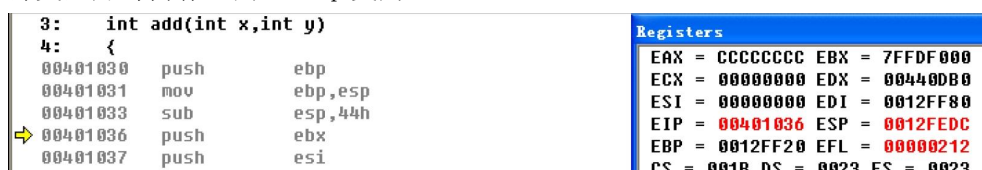
1>从 CALL 指令处按 F11 步入 ADD 函数内部, 观察到 esp 值又小 4 变成了 0012FF24, 这是因为 CALL 指令在执行时首先会把调用点的下一条指令地址入栈, 作为返回地址。



2>按 F11, 执行 push 指令, 将 ebp 寄存器入栈。发现 esp 寄存器值小 4, 说明 ebp 成功入栈。这一步是为了后面恢复栈帧。



3>接下来的 MOV 和 SUB 语句, 把 esp 赋值给 ebp, 实现了栈帧的切换。Esp-44h 是给局部变量开辟内存空间。(esp 变为 0012FEDC)



4>接下来三句 PUSH, 把三个保存寄存器入栈保存。(esp 变为 0012FED0)

```

00401033 sub     esp,44h
00401036 push    ebx
00401037 push    esi
00401038 push    edi

```

```

EIP = 00401039 ESP = 0012FED0
EBP = 0012FF20 EFL = 00000212
CS = 001B DS = 0023 ES = 0023
SS = 0023 FS = 003B GS = 0000

```

5>接下来的四句指令，先把 ebp-44h 即局部变量最顶部地址赋值给 edi，然后用 ecx 计数，循环 11h 次填充赋值，将 44h 的空间全部初始化为 0CCCCCCCch。

Address:	0x0012FED0
0012FED0	80 FF 12 00 00 00 00 00 F0
0012FEDA	FD 7F CC CC CC CC CC CC CC
0012FEE4	CC CC CC CC CC CC CC CC CC
0012FEEE	CC CC CC CC CC CC CC CC CC
0012FEF8	CC CC CC CC CC CC CC CC CC
0012FF02	CC CC CC CC CC CC CC CC CC

6>接下来执行加法运算，ebp+8 访问参数 x，ebp+0ch 访问参数 y。[ebp-4] 存储着临时变量 z，为了保存返回值，将 [ebp-4] 中存储的值赋值给 eax，即最后结果在 eax 中。

```

6:      z=x+y;
0040104F mov     eax,dword ptr [ebp+8]
00401052 add     eax,dword ptr [ebp+0Ch]
00401055 mov     dword ptr [ebp-4],eax
7:      return z;
00401058 mov     eax,dword ptr [ebp-4]

```

Registers	
EAX = 00000004	EBX = 7FFDF000
ECX = 00000000	EDX = 004400B0
ESI = 00000000	EDI = 0012FF20
EIP = 00401058	ESP = 0012FED0
EBP = 0012FF20	EFL = 00000202
CS = 001B	DS = 0023
ES = 0023	SS = 0023
FS = 003B	GS = 0000
OU=0	UP=0
EI=1	PL=0
ZR=0	AC=0
PE=0	CY=0

7>接下来要恢复栈帧。把三个寄存器弹出，esp 变为 0012FEDC。

```

00401046 rep stos dword ptr [edi]
5:      int z=0;
00401048 mov     dword ptr [ebp-4],0
6:      z=x+y;
0040104F mov     eax,dword ptr [ebp+8]
00401052 add     eax,dword ptr [ebp+0Ch]
00401055 mov     dword ptr [ebp-4],eax
7:      return z;
00401058 mov     eax,dword ptr [ebp-4]
8:      }
0040105B pop     edi
0040105C pop     esi
0040105D pop     ebx
0040105E mov     esp,ebp
00401060 pop     ebp
00401061 ret

```

Registers	
EAX = 00000004	EBX = 7FFDF000
ECX = 00000000	EDX = 004400B0
ESI = 00000000	EDI = 0012FF80
EIP = 0040105E	ESP = 0012FEDC
EBP = 0012FF20	EFL = 00000202
CS = 001B	DS = 0023
ES = 0023	SS = 0023
FS = 003B	GS = 0000
OU=0	UP=0
EI=1	PL=0
ZR=0	AC=0
PE=0	CY=0
ST0 = +0.0000000000000000e+0000	
ST1 = +0.0000000000000000e+0000	

Call Stack	
add(int 1, int 3) line 8 + 3 by	
main() line 12 + 9 bytes	
mainCRTStartup() line 206 + 25	

8>接下来两步，MOV 把 ebp 赋值给 esp，POP 把 ebp 弹出，这样就回到了调用 ADD 前的栈帧位置。（esp 变为 0012FF24，即第一步提到的返回地址）。

```

EAX = 00000004 EBX = 7FFDF000
ECX = 00000000 EDX = 004400B0
ESI = 00000000 EDI = 0012FF80
EIP = 00401061 ESP = 0012FF24
EBP = 0012FF80 EFL = 00000202

```

9>最后 ret 指令跳出 call 指令，调用结束。

心得体会：

1. 学会了配置和使用虚拟机。
2. 通过本实验，学习和了解了多个汇编语言的用法。
3. 了解了发生调用时的寄存器操作，对函数的跳转和调用有了更深层面的认知。
4. 通过追踪寄存器存储值、地址空间存储值，对寄存器、局部变量和空间存储有了更深的理解。